

TRACE: Unlocking the Potential of LLMs in Time Series Forecasting for Distributed Energy Resources

Yuxuan Chen  and Haipeng Xie , *Senior Member, IEEE*

Abstract—Accurate power forecasting is essential to the safe operation of power systems. However, the massively distributed energy resources (DERs) data are characterized by both diversity and scarcity. Solving the power forecasting problem requires striking a balance between the model’s generalization ability and its capacity for few-shot learning (FSL). Traditional deep learning approaches are primarily designed for single entities, which limits their generalization capabilities and makes them ineffective in data-scarce scenarios. To address these challenges, we propose TRACE, a novel forecasting framework built on pretrained large language models (LLMs). By tracing and aligning the chain-of-thought (CoT) reasoning patterns, this approach bridges the extensive prior knowledge embedded in LLMs with the unique characteristics of the DERs power forecasting. Moreover, we introduce a multiagent consistency (MaC) framework in TRACE, which achieves thought refinement through collaboration, enhancing the stability of LLMs’ predictions and their ability to handle complex computational tasks, thereby improving prediction accuracy. Experimental results on public real-world datasets demonstrate that TRACE achieves prediction accuracy on par with state-of-the-art deep learning models while significantly outperforming other baseline models in data-scarce scenarios.

Impact Statement—The integration of large-scale DERs into modern power systems presents significant safety risks. Accurate power prediction of DERs is essential for the real-time scheduling of the distribution network, making it particularly important. This article proposes the TRACE framework, which applies generative AI to DERs power forecasting. From an engineering application perspective, this method is well suited to meet the multitier generalization requirements and data scarcity challenges of DERs, contributing to the safe and stable operation of modern power systems. From a technical exploration standpoint, this article proposes a time series forecasting framework based on LLMs. The proposed method outperforms SOTA deep learning approaches in terms of model generalization and FSL capabilities.

Index Terms—Chain-of-thought (CoT), distributed energy resources (DERs), large language models (LLMs), multiagent consistency (MaC), time series forecasting.

I. INTRODUCTION

MODERN distribution networks are distinguished by the integration of large-scale distributed energy resources (DERs). Power forecasting for DERs plays a crucial role in enabling their aggregated management and control, contributing to maintain the safe and stable operation of modern distribution networks [1]. However, the forecasting of large-scale DERs faces the following challenges.

- 1) DERs are defined by their diversity, small scale, and wide geographic distribution. Unlike large-scale centralized resources, they are subject to greater uncertainty, which makes accurate power time series forecasting considerably more difficult.
- 2) Traditional forecasting tasks typically focus on a single entity or a specific scenario. However, when managing a large number of decentralized DERs, forecasting models often require retraining or even replacement with more suitable alternatives [2], leading to poor generalization performance.
- 3) The data associated with DERs is often sparse and incomplete [3], which places their forecasting tasks within the realms of few-shot learning (FSL) and zero-shot learning (ZSL). Consequently, most deep learning methods face significant challenges in achieving satisfactory performance in this context.

Power forecasting for DERs falls under the category of time series forecasting. Solutions to this task can be broadly classified into traditional statistical models and artificial intelligence-based approaches.

Traditional statistical forecasting models are appreciated for their simplicity, strong interpretability, and low computational requirements. However, they face notable limitations when handling sequence data with long-term dependencies and modeling the nonlinear characteristics of the diverse and complex data in distribution networks.

Deep learning-based forecasting models have increasingly become the mainstream approach for time series prediction. These models are highly effective at handling long-term sequences and capturing complex nonlinear patterns in time series data, making them widely applicable in various forecasting scenarios within power systems [17], [18]. However, most deep learning models rely on large volumes of clean data for training, which limits their effectiveness in data-scarce scenarios. Furthermore, forecasting tasks for DERs place high demands on model generalization. Retraining and reconfiguring models for

Received 14 April 2025; accepted 15 May 2025. Date of publication 23 May 2025; date of current version 30 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U24B6010. This article was recommended for publication by Associate Editor Manuel Roveri upon evaluation of the reviewers’ comments. (*Corresponding author: Haipeng Xie.*)

The authors are with the School of Electrical Engineering, Xi’an Jiaotong University, Xi’an 710049, China (e-mail: haipengxie@xjtu.edu.cn).
Digital Object Identifier 10.1109/TAI.2025.3572853

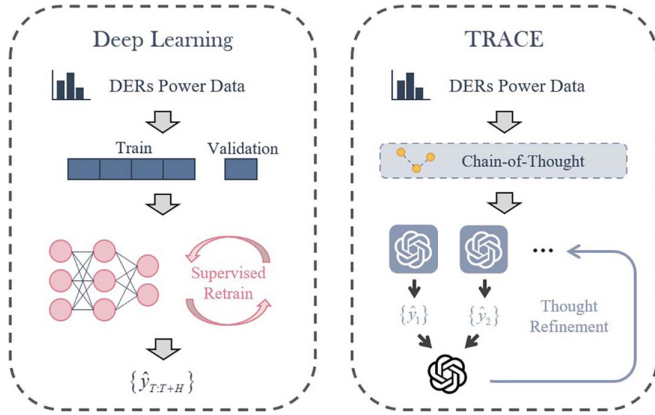


Fig. 1. Comparison between traditional deep learning forecasting process with TRACE.

different entities consume significant computational resources and incur high time costs.

The release of ChatGPT in November 2022 signaled the dawn of the era of generative large-scale models [4]. The application of pretrained LLMs to downstream tasks in specialized domains has since attracted widespread attention from researchers globally. The complex data characteristics, diverse forecasting tasks, and challenges such as data scarcity and incompleteness, arising from a vast number of decentralized entities, make DERs time series forecasting a demanding task that involves a complex combination of robust model generalization and effective FSL capabilities. How to effectively leverage the prior knowledge embedded in LLMs to address DERs time series forecasting tasks remains a critical area for further exploration.

Against this backdrop, this article introduces thought refinement and correction engine (TRACE), a time series forecasting framework for large-scale DERs in distribution networks built on pretrained LLMs, as illustrated in Fig. 1. The main contributions of this article are as follows.

- 1) To the best of our knowledge, the framework proposed in this article, TRACE, represents the first application of multiagent collaboration of LLMs with CoT reasoning to the downstream task of DERs power time series forecasting. Extensive experiments on public real-world datasets demonstrate that TRACE achieves predictive accuracy on par with SOTA deep learning models while excelling in FSL scenarios.
- 2) An instruction fine-tuning method based on CoT reasoning is designed to address the unique characteristics of DERs time series. Through chain-of-thought (CoT) reasoning, the prior knowledge embedded in pretrained LLMs is traced and aligned with the requirements of DERs power time series forecasting tasks.
- 3) This article proposes a multiagent consistency (MaC) framework in TRACE that enables agents to refine and self-correct by referencing the reasoning processes and

answers of “other individuals,” thereby improving prediction accuracy on downstream tasks and effectively reducing inaccuracies caused by hallucinations and biases.

II. RELATED WORK

The massive DERs power data are categorized as time-series data. Predicting their future values is fundamentally a time-series forecasting problem.

A. Traditional Time Series Forecasting Methods

Traditional time-series forecasting methods rely on mathematical and statistical theories, including the autoregressive integrated moving average (ARIMA) [5], seasonal ARIMA (SARIMA), and exponential smoothing (ETS) models [6]. These classic approaches remain widely utilized across various time-series forecasting scenarios [7]. A common approach to point load forecasting involves combining seasonal decomposition with the ARIMA method [8]. By adjusting for seasonal components, this approach significantly enhances the accuracy of load forecasting. In DER power forecasting, these methods can be integrated with uncertainty quantification techniques to effectively improve the adaptability and robustness of forecasting models [9]. Although these classical models can still achieve decent forecasting performance, prediction methods based on traditional mathematical and statistical theories heavily rely on data stationarity and clear seasonal patterns [10], making them inadequate for capturing the nonlinear and complex characteristics of the massive entities in distribution networks.

B. Deep Learning-Based Approaches

Traditional machine learning models (e.g., tree-based models) are highly effective at handling tabular data with feature variables but fall short when it comes to capturing temporal characteristics. Consequently, only deep-learning-based forecasting methods are discussed in this section.

Models based on RNN, such as LSTM [11], Seq2Seq [12], and VAE [13], are particularly effective at handling time-series data due to their ability to capture sequential and contextual dependencies. In recent years, Transformer-based architectures leveraging self-attention mechanisms, along with advanced variants such as Informer [14], Autoformer [15], and FEDformer [16], have increasingly become mainstream approaches for time-series forecasting. Some researchers have employed temporal fusion Transformer models for short-term load forecasting, improving prediction performance by effectively integrating historical and future covariates [17]. Another area of research focuses on enhancing the Transformer architecture and optimizing the multihead attention mechanism, which has been successfully applied to day-ahead photovoltaic power output forecasting [18]. Transformer has achieved notable success in time-series forecasting. However, its capabilities remain limited in certain areas, such as short-sequence forecasting and multitask prediction. It faces significant challenges in handling data scarcity and generalization requirements when applied to massive, multisource entities within distribution networks.

C. LLM-Based Forecasters

Large language models (LLMs), trained on the Transformer architecture with hundreds of millions to trillions of parameters, have demonstrated exceptional performance in fields such as machine translation, task-oriented question answering, and sentiment analysis [19]. Applying LLMs to time-series forecasting represents a cutting-edge and promising area of research, opening up new possibilities for addressing complex temporal prediction challenges. Some researchers have achieved promising results by designing specialized task instructions and prompt to reframe time-series forecasting tasks as text-to-text question-answering problems. This approach has shown success in downstream applications such as pedestrian trajectory [20] prediction and temperature forecasting [21]. However, the aforementioned approaches rely on simple prompts, which fail to fully leverage the extensive prior knowledge embedded in LLMs. Other studies focus on partial parameter fine-tuning of LLMs, utilizing data augmentation techniques [22] and designing specialized embedding patterns [23], [24] tailored for time-series data. However, these methods rely on constructing large-scale time-series datasets to support retraining, which is computationally expensive and lacks the ability to generalize to unseen scenarios.

The aforementioned studies have made exploratory attempts, however, how to align the prior knowledge pretrained in LLMs with the unique characteristics of massive DERs, enabling LLMs to serve as effective forecasters still requires further investigation.

III. METHODOLOGY

A. Problem Statement

This article considers the DERs power forecasting problem as a classic time series forecasting problem. Assume there is historical observation data with t uniform time intervals, l channels

$$\mathbf{X}_{1:t} = \{x_1, x_2, \dots, x_t\} \in l \times t \quad (1)$$

where x_t denotes the observed value at time step t .

The goal is to predict the next h steps observation based on the historical data $\mathbf{X}_{1:t}$

$$\hat{\mathbf{Y}}_{t:t+h} = \{\hat{y}_{t+1}, \hat{y}_{t+2}, \dots, \hat{y}_{t+h} | \mathbf{X}_{1:t}\}. \quad (2)$$

This article addresses power forecasting for various types of DERs, including user multienergy load time series, rooftop photovoltaic output, distributed wind power time series, electric vehicle charging/discharging time series, and user energy storage device charging/discharging time series. The forecasting time scale considered in this study is ultrashort-term (1 step), short-term (6/24 steps), and medium-term (1 week: 168 steps), focusing on both single time series and multivariate time series prediction tasks. The proposed framework, TRACE, operates as an input-output system. Historical observation data are manually input, processed through an internally designed mechanism to ensure MaC, and then output the predicted value.

B. Overall Framework

The proposed TRACE framework is depicted in Fig. 2. The framework consists of three key components.

- 1) *CoT Construction*: A series of CoT is designed based on the unique characteristics of DERs data analysis and power forecasting tasks to enhance the reasoning boundary of LLMs. This process equips LLMs with the ability to align prior knowledge with downstream DERs tasks.
- 2) *Prompt Engineering*: Complete prompt instructions are generated, consisting of a task summary, CoT, and specific task content. These prompts guide multiple agents, each configured with different parameter settings, to output predicted values along with the relevant intermediate reasoning steps.
- 3) *Thought Refinement and Correction*: Finally, a Judge Agent participates in and organizes the thought refinement process among the multiple agents. This ensures MaC by reconciling discrepancies in predictions and reasoning. Ultimately, the Judge Agent provides a coherent and consistent forecasting result.

C. CoT Reasoning

1) *Reasoning Boundary Enhancement*: A recent study suggests that the reasoning capabilities of LLMs are inherently limited and can be quantified using a concept known as the “reasoning boundary” [28]. This reasoning boundary B_{Acc} is a measure related to the complexity of a given task, expressed mathematically by the following formula:

$$B_{\text{Acc}=P_1}(T|A) = \sup_{\text{difficulty}} \{d | \text{Acc}(T|d, A) = P_1\} \quad (3)$$

where P_1 represents the predefined reasoning accuracy threshold for LLMs, T denotes a specific task, and A refers to a particular agent. This formula describes the maximum task complexity achievable when the reasoning accuracy of the LLM for a given task and agent is no less than P_1 .

Additionally, based on the above formula, this study further derived the combination law of reasoning boundary of LLMs when tackling composite tasks

$$B_{\text{Acc}=P_1}^{\text{Combination}}(T_1, T_2, \dots, T_N | A) = \frac{1}{(N-1) \sum_{i=1}^N \frac{c_i}{B_{\text{Acc}=P_1}(T_i | A)} - b_i} \quad (4)$$

where c_i and b_i are scaling factors, and N represents the number of considered tasks.

Building on the above theory, this article puts forward a reasonable hypothesis: the power forecasting problem for DERs is a complex task requiring multistep reasoning and computation. Can LLMs achieve better forecasting performance by optimizing the combination law of reasoning boundary for such problems?

CoT reasoning has been shown to improve the performance of LLMs on complex tasks. Models utilizing CoT have achieved higher scores on various public benchmarks, including sentiment analysis and mathematical computation [29]. In this article, we equip LLMs with CoT reasoning capabilities, allowing

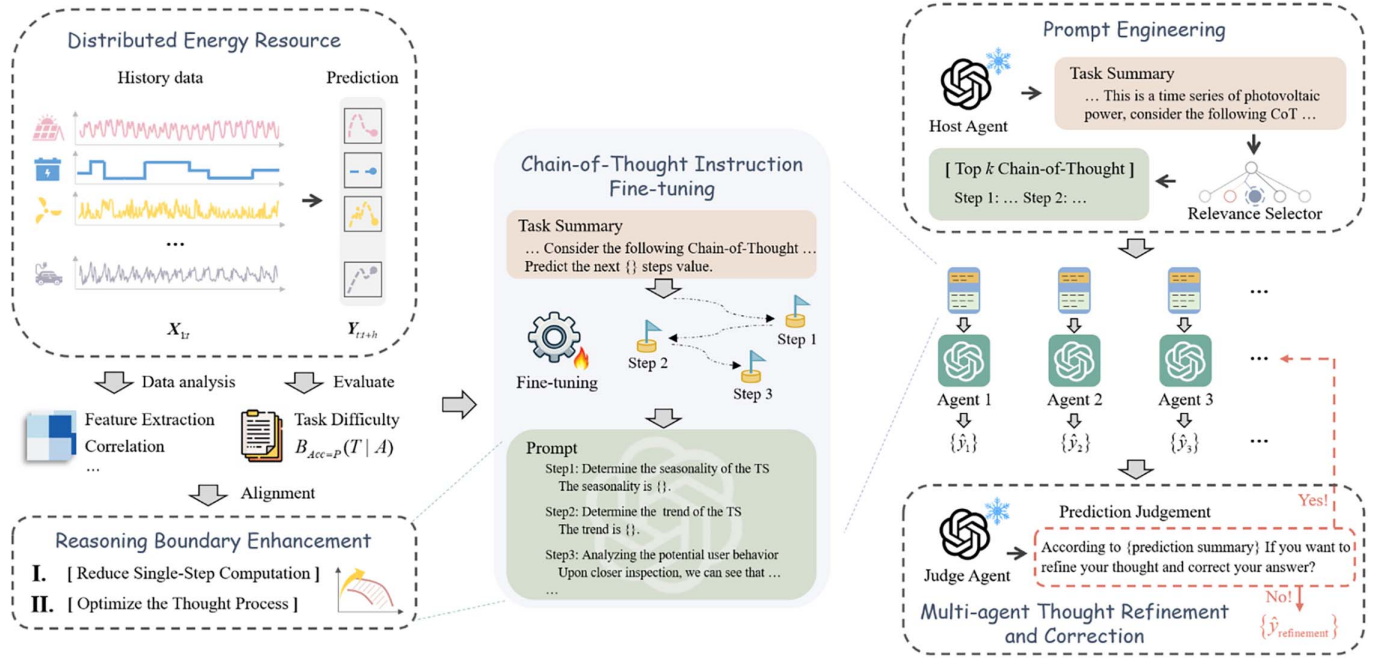


Fig. 2. TRACE framework.

the combination law of reasoning boundary for DERs power forecasting task to be expressed as

$$B_{Acc=P_1}^{DERs-CoT}(T|A) = B_{Acc=P_1}^{Combination}(T_C, T_S|A) = \frac{1}{c_1/(B(C) - b_1) + c_2/(B(S) - b_2)} \quad (5)$$

where T_C and T_S represent calculation reasoning boundary and step planning reasoning boundary, respectively.

Employing CoT reasoning enhances the reasoning boundary in two key aspects: a) *reduce single-step computation*: CoT breaks down large computations into smaller, more manageable ones, thereby enhancing $B(C)$ by adding reasoning boundary. b) *Optimize the thought process*: CoT decomposes complex problem into multiple substeps, reducing task complexity and, consequently, enhancing $B(S)$. Based on the above analysis, adopting CoT reasoning can effectively enhance the reasoning boundary B^{DERs} and therefore achieve accurate power forecasting for DERs.

2) *CoT Instruction Fine-Tuning*: The theoretical derivation above demonstrates that CoT reasoning can enhance the reasoning boundary of LLMs. However, the main challenge is developing an effective CoT strategy tailored to specific tasks. The CoT fine-tuning method proposed in [29] offers only a partial reasoning process before producing the final answer. This training approach fails to help the model develop a genuine “thinking” process and shows limited generalization across tasks. To equip LLMs with genuine reasoning capabilities and align their extensive prior knowledge with the DERs power forecasting tasks, this article designs a series of CoT processes tailored to the specific context of DERs power forecasting, with some CoT examples provided as follows (for more design details, see Section IV-F).

- Consider the temporal decomposition characteristics. Determine whether the time series exhibits seasonality and calculate the seasonal cycle. Assess whether the time series contains a trend and evaluate the magnitude of that trend.
- Take into account the potential user behavior characteristics. Analyze whether the time series reflects underlying user behavior patterns and assess how these behavioral features vary over time.
- Consider the variation characteristics of the time series across different time scales. Evaluate the potential trends in the sequence at specific timestamps (e.g., year, month, date, and weekday) and assess the impact of holidays on these trends.

For a given task, only a limited subset of CoT processes may be relevant. Therefore, the specific CoT elements required for a given task are determined by a relevance selector $CoT - rele(\cdot)$

$$CoT-rele(T) = \{CoT_i\} \\ i \in \text{top-k}(\text{shapley}(CoT, T)) \quad (6)$$

where $\text{shapley}(\cdot, \cdot)$ represents the Shapley contribution of this step to task T , and top-k refers to an *argmax* selecting.

3) *Prompt Engineering*: Prompt engineering combines “summary,” “CoT,” and “content” into a complete prompt. The “summary” defines the role and skills of the LLM, such as: “Consider the following CoT, predict the next value.” The “CoT” section consists of the top-k most relevant steps selected by the relevance selector. The “content” embeds a fixed template for time series analysis. Table I illustrates several prompt engineering processes, including specific CoT contents and summary formats. Through rigorous instruction fine-tuning, we have standardized the output format of the LLMs, as shown on

TABLE I
PROMPT ENGINEERING AND CoT EXAMPLES

Prompt Engineering			Examples of CoT outputs
Prompt 1	Summary	...Consider the following CoT, predict the next {} value.	Upon closer inspection of {type} time series, the seasonality is {}, the trend is {}, the top 3 correlated variables are {variable A} and {variable B} with a correlation of {value}, {variable C} and {variable D} with a correlation of {value}, {variable E} and {variable F} ... My prediction for the next {} value is {0.43, 0.42, 0.41 ...}
	CoT	Step1: Consider the seasonality... Step2: Consider the trend... Step3: Identify the top 3 correlations between variables...	
	Content	From {time} to {time}, the {type} time series is {} ...	
Prompt 2	Summary	...Consider the following CoT, predict the next {} value.	Upon closer inspection of {type} time series, it seems like EVs prefer charging between 11 PM and 7 AM. The trend is {}. The top 3 external factors affecting the variables are: long-term scale impacts {variable A, variable B} with a change of {trend, seasonality}, ... My prediction for the next {} value is {0.43, 0.42, 0.41 ...}
	CoT	Step1: Consider different types of user behaviors... Step2: Consider the trend in charging demand... Step3: Evaluate the impact of external factors on multiple variables.	
	Content	From {time} to {time}, the {type} time series is {} ...	
Prompt 3	Summary	...Consider the following CoT, predict the next {} value.	Upon closer inspection of {type} time series, it seems like user load baseline on {weekend} is {}, user load baseline on {weekday} is {}, user load baseline on {February} is {}, user load baseline on {March} is {}. The seasonality is {12, 24, 168} in specific timescales. My prediction for the next {} value is {0.43, 0.42, 0.41 ...}
	CoT	Step1: Consider variations across different timescales... Step2: Evaluate the potential seasonality in specific timescales... Step3: Combine overall trends and specific granular time scales.	
	Content	From {time} to {time}, the {type} time series is {} ...	
...	...	...	...

the right side of the table. It can be observed that for specific tasks and CoT steps, the LLMs' outputs consistently follow a fixed format, which facilitates the accurate extraction of key information for subsequent processing.

D. MaC in TRACE

LLMs have limited reasoning capabilities when it comes to complex time-series tasks [22], making them more prone to generating flawed thought processes and conclusions. Additionally, since their outputs are determined based on probabilistic rankings rather than logical validation, it is difficult to eliminate issues such as model hallucinations and inherent biases [25].

A salient characteristic of humanity is that people learn from each other. A recent study suggests that LLMs can be regarded as weak learners and explores the potential of combining them with ensemble learning [26]. Inspired by this, this article raises a question: can LLMs refine their reasoning process by learning from the thoughts of other agents, ultimately arriving at better answers?

In this context, this article proposes MaC framework designed to implement a thought refinement and correction process for LLMs through cooperative reasoning.

The thought refinement process consists of multiple iterative rounds. In each round, the Judge Agent aggregates the outputs of all predictor agents and generates n prediction summaries. Each summary includes the mean of all agents' predictions and intermediate statistical metrics. These summaries are then fed back to the respective predictor agents, allowing each agent to observe the statistical mean of the other agents' predictions. Based on this feedback, each agent refines its predictions and generates a new output. The process continues until the

consistency score falls below a predefined threshold or the maximum number of iterations is reached. At this point, the Judge Agent outputs the final prediction. The pseudocode for the thought refinement process is provided in Algorithm 1 and Fig. 3.

In the MaC framework, we further integrated a Factuality Verification module. Following the methodology proposed by Feng et al. [27], a scoring metric $\text{Score}_{\text{fact}}$ was introduced to assess the factual accuracy of the generated summaries

$$\text{Score}_{\text{fact}} = \text{sum} - \text{fact}(S_{\text{Judge}} | \sum_{i=1}^K R_{\text{Agent-}i}) \in [0, 1] \quad (7)$$

where S_{Judge} and $R_{\text{Agent-}i}$ represent the summary generated by the Judge Agent and the reasoning content of the i th agent, respectively.

We consider three metrics: *lexical overlap*, *semantic consistency*, and *factual consistency*, the score of factuality can be calculated as

$$\text{sum-fact}(\cdot) = \frac{s^{lo} + s^{sc} + s^{fc}}{3}. \quad (8)$$

Finally, we allow the Judge Agent to generate multiple summaries and select the one with the highest score as the final judgment. This is intended to ensure that all agents receive accurate statistical information, preventing information bias from misleading their reasoning processes.

IV. EXPERIMENTS

In this section, we evaluate TRACE forecasting framework and answer the following questions: 1) Are LLMs capable of accurately predicting the power of DERs?; 2) can LLMs

Algorithm 1: Thought Refinement Process in TRACE Framework**Inputs:**

n : Number of LLM Agents.
 k : Number of task-related CoT steps.
 $\mathcal{T}$: Task description (e.g., “This is a time series of photovoltaic power”)
 $\mathcal{C}$: Set of task-related CoT steps, $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.
 ε : Consistency threshold for convergence.

Output:

y_{final} : Final prediction results.

Procedure:**1. Initialization**

Generate task prompt: $\mathcal{P} = \mathcal{T} \cup \mathcal{C}$.

Each Agent i generates predictions and intermediate results based on $\mathcal{P}$: $y_i^{(1)}, m_i^{(1)}$

2. Iterative Process

for $r=1$ **to** $\text{Round}_{\text{max}}$ **do:**

1) Judge Agent Summary

Compute mean prediction and mean intermediate results:

$$\bar{y}^{(r)} = \frac{1}{n} \sum_{i=1}^n y_i^{(r)}, \quad \bar{m}^{(r)} = \frac{1}{n} \sum_{i=1}^n m_i^{(r)}$$

Generate prediction summary $\mathcal{S}_i^{(r)} = \{\bar{y}^{(r)}, \bar{m}^{(r)}, \text{Consensus Summary}\}$ for each Agent i .

2) Consistency Check

Compute consistency score:

$$\text{Consistency}^{(r)} = \frac{1}{n} \sum_{i=1}^n (y_i^{(r)} - \bar{y}^{(r)})^2$$

if $\text{Consistency}^{(r)} < \varepsilon$ **then:**

Generate final prediction: $y_{\text{final}} = \bar{y}^{(r)}$

break

3) Thought Refinement

Each Agent i refines its predictions and intermediate results based on $\mathcal{S}_i^{(r)}$:

$$\text{Agent}_i\text{'s Output: } \{y_i^{(r+1)} \mid y_i^{(r)}, \mathcal{S}_i^{(r)}\}, \{m_i^{(r+1)} \mid m_i^{(r)}, \mathcal{S}_i^{(r)}\}$$

3. Final Output

Generate final prediction: $y_{\text{final}} = \bar{y}^{(r)}$

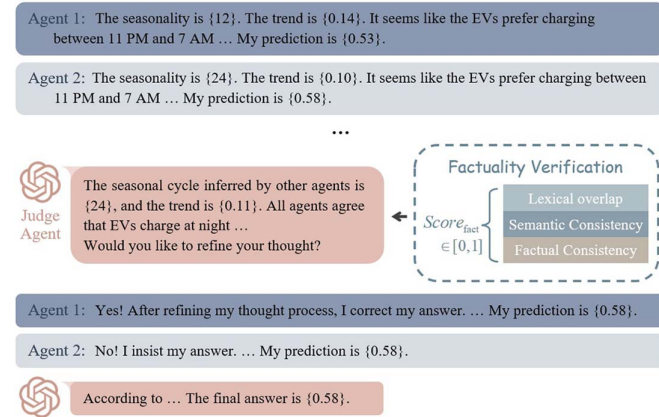


Fig. 3. Thought refinement and correction process.

perform effectively in FSL scenarios?; and 3) to what extent does TRACE enhance the predictive performance of LLMs in DERs time series forecasting scenarios?

A. Datasets

The publicly available dataset [30], [31], [32], [33] used in this study includes various types of DERs. These comprise user-level multienergy load data, distributed photovoltaic systems, distributed wind power, the charging and discharging power of EVs, and the charging and discharging power of user-level

energy storage devices. The detailed description of these datasets is shown in Table II. We standardized all datasets to a two-year time series. Outliers were handled using two common methods: the 3σ rule and the interquartile range (IQR) criterion. For normalization, we applied the following method:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x} - \min(\mathbf{x})}{\max(\mathbf{x}) - \min(\mathbf{x})} \quad (9)$$

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}}{\max(|\max(\mathbf{x})|, |\min(\mathbf{x})|)} \quad (10)$$

where the EVs and storage datasets were normalized to the range of $[-1, 1]$ using (10) with positive values represent charging and negative values represent discharging. The remaining datasets were normalized to the range of $[0, 1]$ using the min-max method in (9).

B. Implementation Details

The LLMs used in this study include “Llama-3.1-70B-Instruct” and “gpt-4o.” Among them, the Host Agent and Judge Agent are implemented using “GPT” models, while the “Llama” models serve as forecasters. The “top_p” parameter for all models is set to 0.9, with the “temperature” parameter for “GPT” models set to 0.8, for “Llama” uniformly distributed between 0.5 and 0.9. For deep learning models, we implement a unified set of training hyperparameters across all datasets to ensure consistency. Specifically, we trained all models for ten epochs with early stopping (patience = 5) based on validation

TABLE II
DETAILED DESCRIPTION OF DATASETS

Source	Dataset	Description	Features	Time Resolution
NEST Platform [30] (Swiss Federal Laboratories for Materials Science and Technology)	ELE	Building electricity load data	42	1 hour
	HAC	Building heating and cooling load data	28	1 hour
	PV	Distributed photovoltaic power data	20	1 hour
China Longyuan Power Group Corporation [31]	WP	Wind power data	16	30 minutes
German Institute for Economic Research [32]	EV1	Electric vehicle charging and discharging data	11	1 hour
	EV2		11	1 hour
	EV3		11	1 hour
	EV4		11	1 hour
International Energy Research Centre (IERC) [33]	STO	User-level energy storage data	6	1 hour

TABLE III
UNIVARIATE PREDICTION RESULTS

		TRACE		DLinear		FEDformer		Autoformer		Informer		SARIMA		ETS		ARIMA	
		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
ELE	1	0.038	0.057	0.029	0.036	0.033	0.051	0.054	0.089	0.055	0.092	0.073	0.091	0.269	0.389	0.074	0.133
	6	0.050	0.084	0.060	0.088	0.144	0.189	0.129	0.192	0.148	0.235	0.148	0.222	0.287	0.433	0.170	0.294
	24	0.081	0.129	0.198	0.277	0.237	0.324	0.240	0.366	0.241	0.349	0.312	0.452	0.358	0.498	0.307	0.463
	168	0.189	0.305	0.241	0.319	0.288	0.386	0.285	0.411	0.299	0.404	0.360	0.527	0.412	0.555	0.354	0.506
	Avg	0.090	0.144	0.132	0.180	0.176	0.237	0.177	0.265	0.186	0.270	0.223	0.323	0.332	0.469	0.226	0.349
HAC	1	0.026	0.034	0.010	0.014	0.012	0.018	0.015	0.024	0.018	0.028	0.087	0.135	0.202	0.323	0.088	0.142
	6	0.038	0.053	0.077	0.105	0.116	0.153	0.128	0.199	0.130	0.202	0.145	0.239	0.258	0.377	0.169	0.240
	24	0.052	0.084	0.136	0.188	0.165	0.210	0.160	0.231	0.181	0.276	0.272	0.355	0.376	0.494	0.271	0.348
	168	0.209	0.361	0.208	0.269	0.227	0.285	0.244	0.337	0.253	0.344	0.340	0.457	0.389	0.522	0.356	0.461
	Avg	0.081	0.133	0.108	0.144	0.130	0.167	0.137	0.198	0.146	0.213	0.211	0.297	0.306	0.429	0.221	0.298
WP	1	0.085	0.152	0.119	0.211	0.099	0.190	0.090	0.167	0.106	0.179	0.179	0.239	0.186	0.326	0.158	0.302
	6	0.096	0.164	0.128	0.220	0.140	0.199	0.158	0.213	0.151	0.239	0.190	0.266	0.265	0.370	0.247	0.376
	24	0.145	0.238	0.152	0.227	0.186	0.234	0.197	0.256	0.222	0.298	0.312	0.398	0.394	0.505	0.287	0.405
	168	0.195	0.297	0.240	0.315	0.294	0.375	0.292	0.371	0.335	0.431	0.365	0.488	0.441	0.534	0.389	0.490
	Avg	0.130	0.213	0.160	0.243	0.180	0.250	0.184	0.252	0.203	0.287	0.262	0.348	0.321	0.434	0.270	0.393
PV	1	0.033	0.048	0.009	0.014	0.012	0.017	0.012	0.018	0.017	0.026	0.055	0.082	0.078	0.148	0.058	0.097
	6	0.050	0.092	0.053	0.092	0.062	0.119	0.093	0.154	0.131	0.199	0.073	0.111	0.104	0.177	0.070	0.142
	24	0.099	0.138	0.265	0.512	0.188	0.324	0.204	0.386	0.248	0.287	0.180	0.330	0.245	0.385	0.160	0.269
	168	0.301	0.569	0.477	0.779	0.450	0.688	0.538	0.818	0.499	0.756	0.288	0.414	0.371	0.505	0.305	0.488
	Avg	0.121	0.212	0.201	0.349	0.178	0.287	0.212	0.344	0.224	0.317	0.149	0.234	0.200	0.304	0.148	0.249
EV1	1	0.011	0.020	0.022	0.029	0.034	0.053	0.048	0.074	0.048	0.070	0.150	0.217	0.168	0.259	0.142	0.246
	6	0.030	0.058	0.066	0.095	0.082	0.143	0.090	0.168	0.099	0.165	0.254	0.355	0.245	0.388	0.262	0.397
	24	0.105	0.232	0.179	0.280	0.210	0.366	0.254	0.433	0.277	0.410	0.273	0.380	0.331	0.486	0.308	0.459
	168	0.276	0.417	0.301	0.495	0.359	0.585	0.404	0.721	0.391	0.678	0.439	0.576	0.445	0.620	0.367	0.541
	Avg	0.106	0.182	0.142	0.225	0.171	0.287	0.199	0.349	0.204	0.331	0.279	0.382	0.297	0.438	0.270	0.411
EV2	1	0.054	0.093	0.044	0.065	0.038	0.056	0.041	0.061	0.066	0.112	0.350	0.501	0.434	0.522	0.420	0.545
	6	0.098	0.143	0.123	0.177	0.111	0.182	0.134	0.194	0.146	0.243	0.367	0.532	0.447	0.545	0.417	0.538
	24	0.187	0.288	0.200	0.279	0.198	0.307	0.279	0.430	0.283	0.445	0.358	0.516	0.459	0.577	0.449	0.589
	168	0.355	0.490	0.366	0.517	0.390	0.588	0.451	0.792	0.414	0.683	0.409	0.575	0.453	0.568	0.453	0.601
	Avg	0.174	0.254	0.183	0.260	0.184	0.283	0.226	0.369	0.227	0.371	0.371	0.531	0.448	0.553	0.435	0.568
EV3	1	0.041	0.062	0.047	0.072	0.048	0.072	0.055	0.095	0.057	0.108	0.381	0.525	0.489	0.570	0.390	0.488
	6	0.053	0.176	0.115	0.170	0.101	0.177	0.119	0.194	0.103	0.186	0.247	0.398	0.455	0.539	0.406	0.513
	24	0.141	0.277	0.208	0.313	0.224	0.405	0.253	0.386	0.248	0.377	0.391	0.544	0.430	0.517	0.438	0.569
	168	0.280	0.448	0.296	0.545	0.402	0.729	0.390	0.703	0.422	0.721	0.412	0.583	0.484	0.568	0.470	0.631
	Avg	0.129	0.241	0.167	0.275	0.194	0.346	0.204	0.345	0.208	0.348	0.358	0.513	0.465	0.549	0.426	0.550
EV4	1	0.070	0.121	0.067	0.104	0.062	0.115	0.067	0.111	0.066	0.105	0.444	0.566	0.406	0.506	0.473	0.576
	6	0.126	0.194	0.139	0.208	0.133	0.222	0.138	0.237	0.129	0.204	0.452	0.581	0.443	0.573	0.469	0.559
	24	0.200	0.315	0.245	0.322	0.268	0.404	0.286	0.432	0.255	0.385	0.455	0.588	0.458	0.597	0.481	0.598
	168	0.348	0.486	0.377	0.530	0.429	0.660	0.477	0.735	0.461	0.680	0.476	0.610	0.450	0.585	0.485	0.614
	Avg	0.186	0.279	0.207	0.291	0.223	0.350	0.242	0.379	0.228	0.344	0.457	0.586	0.439	0.565	0.477	0.587
STO	1	0.067	0.106	0.060	0.090	0.057	0.098	0.061	0.097	0.066	0.111	0.140	0.208	0.166	0.331	0.143	0.234
	6	0.081	0.169	0.099	0.170	0.118	0.175	0.127	0.208	0.145	0.211	0.188	0.256	0.229	0.476	0.202	0.312
	24	0.154	0.287	0.188	0.295	0.226	0.290	0.233	0.374	0.266	0.385	0.265	0.330	0.283	0.553	0.254	0.387
	168	0.225	0.360	0.285	0.413	0.354	0.476	0.371	0.589	0.402	0.582	0.434	0.584	0.458	0.686	0.376	0.589
	Avg	0.132	0.231	0.158	0.242	0.189	0.260	0.198	0.317	0.220	0.322	0.257	0.345	0.284	0.512	0.244	0.381

Note: The bold entries indicate the best performance achieved among the compared methods in each respective evaluation metric.

loss, using the Adam optimizer with an initial learning rate of $1e^{-4}$, adjusted via linear warmup for 5% of steps followed by cosine decay. The batch size is set to 64. All experiments were conducted on a server featuring 4 Intel Xeon Gold 6254 processors and 2 NVIDIA A800 GPU. It is worth noting that no additional retraining was performed on LLMs with accessible data, only instruction fine-tuning was applied.

C. Evaluation Metrics

This study employs two error metrics, MAE and RMSE, to evaluate prediction performance

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \quad (11)$$

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}. \quad (12)$$

D. Main Results

The baseline models selected in this study include traditional mathematical statistical models such as ARIMA, ETS, and SARIMA, as well as SOTA deep learning models including Informer, Autoformer, FEDformer, and DLinear. Univariate prediction results with prediction horizon $h = 1, 6, 24, 168$ are presented in Table III. The best results are highlighted in bold. It is evident that the proposed TRACE framework achieves SOTA performance, attaining the lowest average error across all

TABLE IV
MULTIVARIATE PREDICTION RESULTS

	TRACE		DLinear		FEDformer		Autoformer		Informer	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
ELE	0.108	0.175	0.133	0.205	0.144	0.248	0.162	0.275	0.179	0.288
HAC	0.162	0.247	0.157	0.249	0.178	0.304	0.189	0.314	0.196	0.337
WP	0.098	0.180	0.156	0.193	0.151	0.186	0.175	0.248	0.188	0.272
PV	0.126	0.229	0.188	0.326	0.217	0.429	0.205	0.386	0.220	0.444
EV1	0.155	0.323	0.170	0.355	0.210	0.573	0.233	0.561	0.259	0.574
EV2	0.158	0.339	0.185	0.388	0.212	0.572	0.235	0.570	0.271	0.577
EV3	0.172	0.365	0.174	0.387	0.225	0.588	0.211	0.585	0.245	0.596
EV4	0.140	0.291	0.153	0.362	0.193	0.585	0.222	0.597	0.243	0.621
STO	0.280	0.558	0.366	0.631	0.444	0.763	0.416	0.722	0.465	0.790

Note: The bold entries indicate the best performance achieved among the compared methods in each respective evaluation metric.

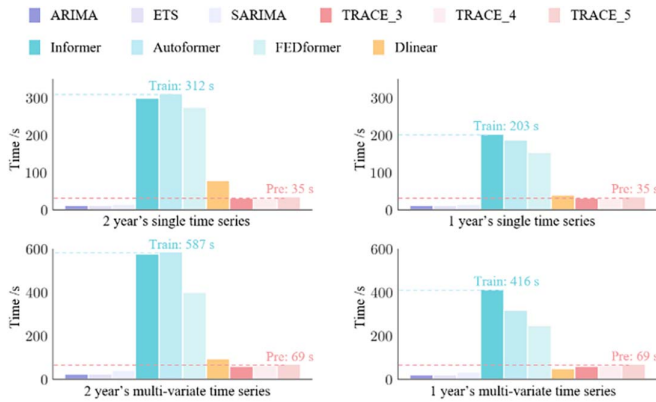


Fig. 4. Average time consumption of each model.

real-world time series datasets. The improvements of TRACE are not only observed in datasets with prominent temporal characteristics, such as load datasets, but also in complex datasets such as wind power and electric vehicle datasets, where it consistently demonstrates low prediction errors. These results highlight the potential of LLMs in DERs power forecasting.

Table IV presents the results of multivariate time series prediction, where all values are averaged over prediction horizons of $h = 1, 6, 24$ steps. It can be observed that the TRACE framework demonstrates the capability to predict multivariate time series, achieving SOTA performance across eight datasets.

Additionally, the training time consumption of each model was evaluated, as shown in Fig. 4. For LLMs, the reported time refers to reasoning time, while “TRACE_i” represents the collaborative framework with i agents. The results indicate that deep learning models require extensive retraining time when faced with new entities, leading to limited flexibility. Although traditional statistical models have shorter training times, their prediction accuracy is relatively low. Compared to deep learning models, although LLM-based models require more time for a single prediction, they eliminate the need for retraining on new subjects, making them a time-efficient solution overall, offering excellent generalization capability while maintaining low forecasting cost requirements.

E. TRACE: Superior Performance in Data-Scarce Scenarios

The power data of massive DERs often exhibit characteristics of data scarcity and incompleteness. Therefore, it is essential

to investigate the predictive performance of TRACE in FSL and ZSL scenarios. This section establishes three experimental groups to evaluate the prediction performance of each model in the following scenarios: 1) FSL scenario; 2) FSL scenario with varying levels of data scarcity; and 3) the scenario with different levels of data missingness.

The prediction error distributions for each model across nine FSL datasets are shown in Fig. 5. It is clear that all models experience a decrease in prediction accuracy in the data scarcity scenario, accompanied by a larger variance in prediction errors. Deep learning-based models are most adversely affected by data scarcity, followed by traditional statistical models. This is because deep learning models rely on large amounts of data to train complex model structures and are more prone to overfitting when data are limited. In contrast, ARIMA and SARIMA models have simpler structures, and once the dataset length exceeds the significant trend and seasonal cycles, the prediction accuracy of these models approaches their performance limit. The proposed TRACE prediction framework exhibits the smallest prediction errors across all datasets, thereby demonstrating exceptional prediction robustness in the data scarcity scenario.

The prediction errors of each model under different levels of data scarcity are shown in Fig. 6 (left), where the numbers in the figure represent the sample size. As the number of accessible samples increases, the prediction accuracy of deep learning-based models gradually improves. TRACE, on the other hand, performs consistently well across various FSL scenarios, as it does not require additional training but instead leverages existing prior knowledge for reasoning.

The prediction errors of each model under the data missingness scenario, simulated through random deletion, are shown in Fig. 6 (right). It should be noted that models requiring retraining cannot complete the prediction task on data with missing values. Therefore, in this experiment, the remaining models were tested on data processed using forward fill (ffill). It can be observed that the prediction errors of these models increase as the missing data ratio rises. The TRACE framework proposed in this article offers two key advantages: 1) it can directly output predictions without requiring additional missing data handling; and 2) it demonstrates strong prediction stability under a certain level of data missingness, with only a slight decline in prediction accuracy.

Additionally, for scenarios with missing data, we conducted supplementary experiments to evaluate TRACE’s time series

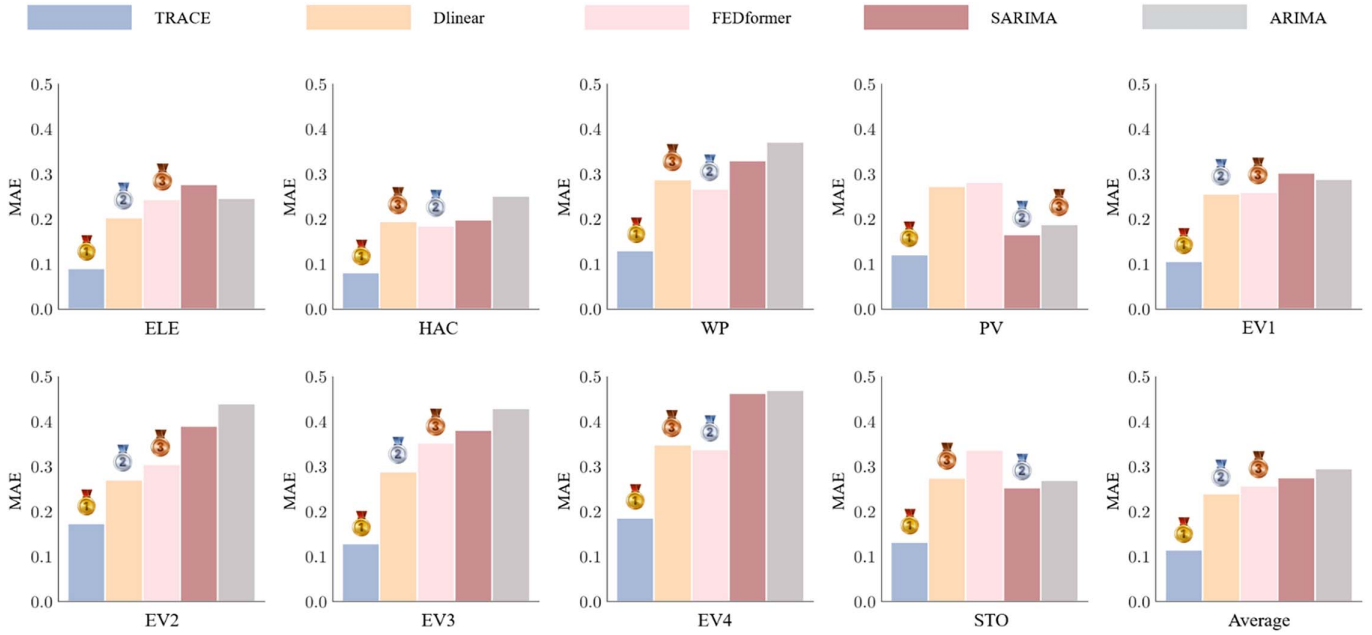


Fig. 5. Prediction errors of different models on FSL scenario.

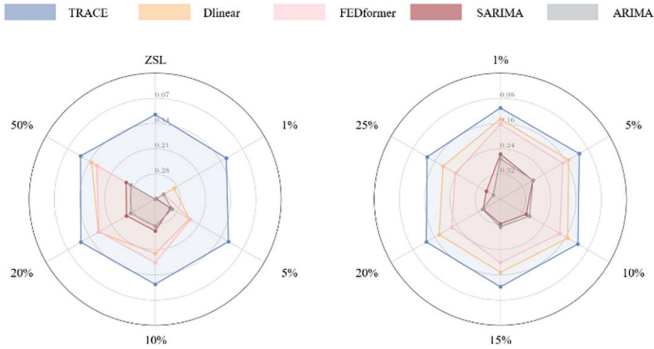


Fig. 6. Prediction errors under different levels of data scarcity (left) and data missingness (right).

imputation capability. No modifications were made to the CoT or MaC structures, only “task description” was altered (e.g., “...please impute the missing data”).

First, we aimed to evaluate the performance of TRACE against other commonly used imputation methods. The selected baseline models include two traditional methods: Linear Interpolation and K-nearest neighbors (KNNs), and two SOTA generative models, namely PSA-GAN [37] (based on GAN) and diffusion-TS [38] (based on diffusion models). We did not include RNN-based or Transformer-based imputation models because their imputation principles are fundamentally prediction based, and the predictive performance of such models has already been extensively demonstrated in other experimental results of this article. The imputation MAE for all models is presented in Table V, all results are averaged over missing ratio of 5%, 10%, 15%, and 20%. As shown, TRACE achieves SOTA imputation performance due to its exceptional FSL capabilities.

To further assess the practical utility of TRACE as an imputation model, we conducted additional experiments to

TABLE V
IMPUTATION MAE

	TRACE	Linear Interpolation	KNN	PSA-GAN	Diffusion-TS
ELE	0.113	0.179	0.175	0.122	0.115
HAC	0.117	0.185	0.188	0.133	0.108
WP	0.136	0.202	0.174	0.175	0.179
PV	0.127	0.191	0.186	0.128	0.144
EV1	0.120	0.198	0.189	0.159	0.131
EV2	0.188	0.276	0.280	0.293	0.192
EV3	0.135	0.228	0.225	0.221	0.166
EV4	0.192	0.280	0.267	0.250	0.199
STO	0.144	0.203	0.202	0.194	0.157
Avg	0.141	0.216	0.210	0.186	0.155

Note: The bold entries indicate the best performance achieved among the compared methods in each respective evaluation metric.

evaluate the impact of data imputed by different models on the training of deep learning models. We selected two high-performing deep learning models, DLinear and FEDformer, and trained them on datasets imputed by different imputation methods. The resulting prediction MAE on the test set is shown in Fig. 7. The experimental group using data imputed by TRACE exhibited the lowest final prediction MAE, validating TRACE’s ability to effectively fill scarce data.

F. Ablation Study

1) *Data Analysis and CoT Construction*: The CoT reasoning is pivotal for effectively utilizing the extensive prior knowledge embedded in LLMs to address the power forecasting challenges of DERs. In this section, a data analysis of DERs time series is performed to inform the design of thought processes tailored to the specific task characteristics, thereby validating the rationality of the proposed CoT approach.

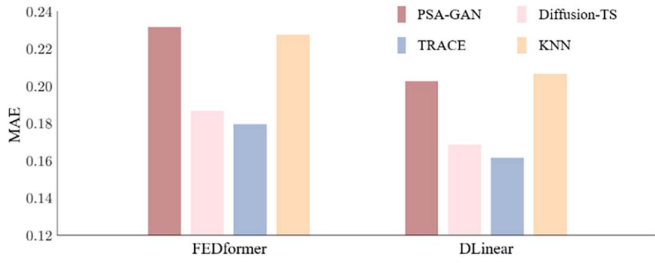


Fig. 7. Prediction MAE of deep models using various imputed data.

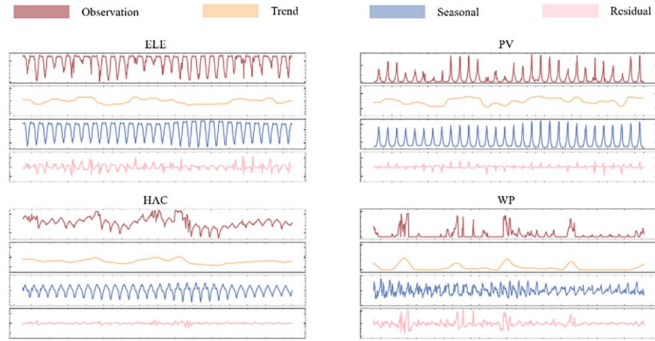


Fig. 8. STL decomposition of DERs time series.

Many DERs power time series, such as load, photovoltaic output, and wind power output, exhibit distinct characteristics of time series decomposition. By applying the STL decomposition method, these time series can be divided into trend, seasonal, and residual components, as illustrated in Fig. 8. For time series such as load and PV output, which display prominent seasonal components, the thought process should prioritize seasonal factors. On the other hand, for time series such as wind power, which exhibits relatively weaker seasonal components but stronger trend components, both factors should be considered comprehensively in the thought process. In summary, the CoT template constructed in this study incorporates steps designed to address the decomposition characteristics of most DERs time series, ensuring that the reasoning steps aligns with the inherent features of DERs data.

Certain fixed-pattern changes observed in time series, such as those of load and electric vehicles, may be attributed to specific user behaviors. These behavioral patterns represent prior knowledge that LLMs encounter during their pretraining process. Fig. 9 illustrates the distribution of charging times for four types of electric vehicle users, based on clustering of the electric vehicle dataset, highlighting distinct user behavior patterns. For instance, type 1 electric vehicle users primarily charge their vehicles at night, while type 4 users prefer daytime charging. Type 2 users tend to charge in the evening and late at night after work, whereas type 3 users typically keep their vehicles charging during daytime working hours. Therefore, incorporating an analysis of potential fixed user behavior patterns exhibited by the time series when constructing the CoT can enhance the accuracy of DERs power forecasting.

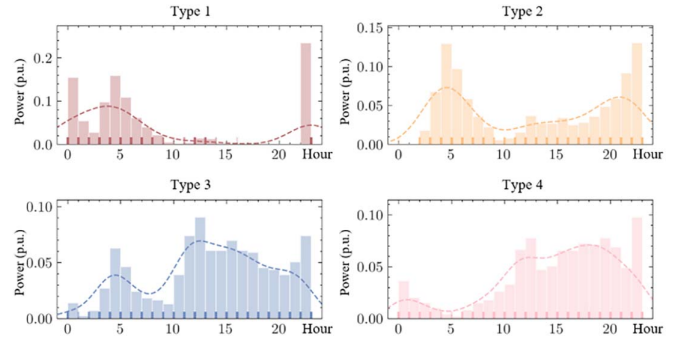


Fig. 9. Charging times distribution of four types of EVs.

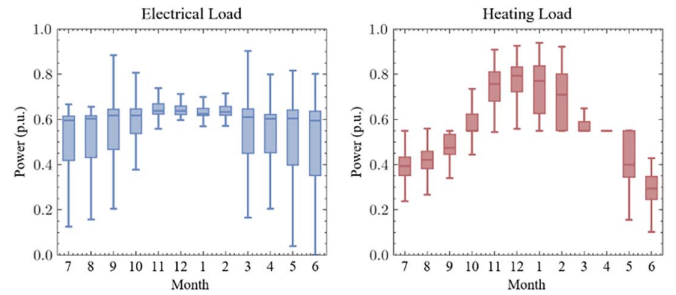


Fig. 10. Baseline variations of electric load and heating load on monthly time scale.

The time series of DERs such as load and photovoltaic output exhibit baseline variations on specific time scales. Fig. 10 demonstrates these baseline variations for two types of user loads (electric load and heating load) on a monthly time scale. It is evident that the variation in electric load is primarily reflected in its variance, with smaller variance observed in winter and larger variance in summer. In contrast, the variation in heating load is mainly reflected in its mean value, with winter heating loads being higher than those in summer. This suggests that, when constructing CoTs, it is valuable to prompt LLMs to analyze the baseline variations of specific DERs entities at a given time scale. Additionally, leveraging the model's prior knowledge of social phenomena, such as the impact of holidays, is important. This is particularly useful, as deep learning models often struggle to capture such knowledge.

2) *CoT Enhances the Forecasting Capabilities of LLMs:* This section validates the effectiveness of the proposed CoT reasoning in enhancing the forecasting capabilities of LLMs across various complex DERs scenarios. Table VI presents a comparative experiment in which specific components are removed from the prompt. "Promptcast" refers to the simple prompting method used in [21], which only provides the start and end times of the time series along with the values.

It can be observed that removing the CoT leads to an increase in forecasting MAE errors across all datasets, with a more significant rise in datasets such as EVs, while the increase is relatively smaller in datasets such as wind power. This may be due to the fact that the CoT designed in this article is particularly effective for time series with more pronounced

TABLE VI
CHANGES IN MAE UNDER DIFFERENT PROMPTS
COMPARED TO TRACE

	w/o CoT	w/o summary	Promptcast
ELE	+0.051	+0.004	+0.077
HAC	+0.040	+0.004	+0.042
WP	+0.024	+0.003	+0.037
PV	+0.057	+0.002	+0.041
EV1	+0.025	+0.001	+0.024
EV2	+0.063	+0.004	+0.076
EV3	+0.061	+0.001	+0.080
EV4	+0.076	+0.002	+0.195
STO	+0.050	+0.003	+0.052
Average	+0.049	+0.003	+0.069

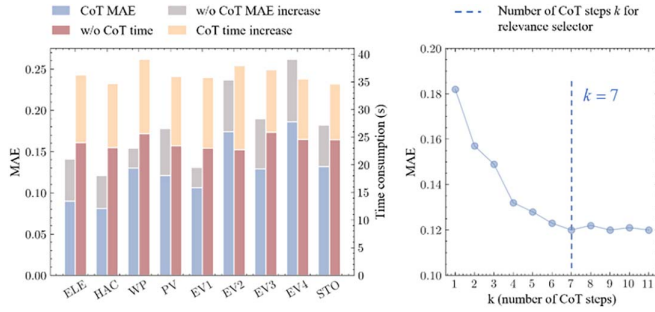


Fig. 11. MAE and time consumption of CoTs (left); variation of MAE with the number of CoT steps k (right).

trends, seasonality, user behavior patterns, and other similar characteristics. After removing the task summary, there is no significant change in the MAE, indicating that the task description and role definition have little impact on the ability of LLMs to handle numerical prediction tasks. Promptcast, which employs simple prompting, results in the highest MAE.

The use of CoT reduces the MAE prediction errors but also results in increased time consumption, as shown in Fig. 11 (left). Although using CoT increases the time by approximately one-third, TRACE still remains a low-cost time option compared to retraining deep learning models. Fig. 11 (right) illustrates the effect of the number of CoT steps selected by the relevance selector on prediction error. It can be observed that as k increases, the prediction error decreases, but a certain degree of diminishing returns is also evident. In the main experimental setting of this article, k is set to 7.

Another advantage of CoT reasoning is its significant enhancement of interpretability in forecasting tasks. Conventional deep learning models operate as black boxes that fail to reveal the factors influencing decisions, whereas TRACE provides explicit reasoning steps that clearly demonstrate which elements contribute decisively to predictions. While traditional deep models learn knowledge implicitly, TRACE actively embeds domain knowledge into the CoT framework. Regarding anomaly detection, even well-trained deep neural networks risk generating unphysical outputs (e.g., negative load values). In contrast, TRACE's integration of prior knowledge constraints and thought refinement processes ensures physically plausible

TABLE VII
MAE UNDER DIFFERENT COOPERATIVE FRAMEWORKS

	TRACE	W/o MaC	Self-Consistency	MaD	Bagging
ELE	0.090	0.099	0.092	0.093	0.094
HAC	0.081	0.083	0.085	0.086	0.093
WP	0.130	0.177	0.155	0.123	0.132
PV	0.121	0.126	0.130	0.128	0.128
EV1	0.106	0.117	0.122	0.120	0.123
EV2	0.174	0.187	0.201	0.169	0.174
EV3	0.129	0.144	0.130	0.148	0.144
EV4	0.186	0.194	0.179	0.185	0.178
STO	0.132	0.152	0.138	0.142	0.138
Average	0.128	0.142	0.137	0.133	0.134
FSL	0.132	0.155	0.052	0.134	0.060
missingness	0.149	0.165	0.071	0.150	0.063

Note: The bold entries indicate the best performance achieved among the compared methods in each respective evaluation metric.

TRACE

Judge Agent: According to other agents' prediction, the average seasonal cycle inferred by other agents is 24, and the trend is 0.04. All agents agree that the EVs prefer charging between 11 PM and 8 AM. Please check the seasonality you calculated, as well as the user behavior patterns of EVs. Would you like to refine your thought?

Agent 1: Yes! After refining my thought process, I correct my answer. The seasonality is 24. The trend is 0.04. It seems like the EVs prefer charging between 11 PM and 8 AM. My prediction is **0.43**. ✓

Multi-agent Debate

Agent 1: The seasonality is 12. The trend is 0.03. It seems like the EVs prefer charging between 11 PM and 7 AM. My prediction is 0.53.

Agent 2: The seasonality is 24. The trend is 0.04. It seems like the EVs prefer charging between 11 PM and 8 AM. My prediction is 0.43.

User Input: Using the solutions from other agents as additional information, can you give an updated response

Agent 1: Given the information provided in other agents' solutions, I updated my response. The seasonality is 24. The trend is 0.03. It seems like the EVs prefer charging between 11 PM and 7 AM. My prediction is **0.50**. ✗

Fig. 12. Thought refinement in TRACE versus multiagent debate.

predictions without such violations. A complete TRACE's reasoning process can be found in Appendix A.

3) *MaC Further Enhances the Prediction Accuracy of LLMs:* In this section, we further explore how MaC improves the performance of LLMs in complex DERs power prediction tasks. Table VII presents the prediction errors when MaC is removed and compares with other similar frameworks. Specifically, "self-consistency [34]" involves making multiple predictions with the same LLM and taking the mode as the final result; "multiagent debate [35]" (MaD) directly copies other LLMs' outputs to the current model, which then output final prediction; Bagging follows the ensemble learning approach, averaging the predictions from multiple LLMs.

It can be observed that the MAE increases significantly after removing MaC. Among the other similar frameworks, multiagent debate performs second only to TRACE. The MaC framework proposed in this article, by introducing the Judge Agent, can accurately point to the specific CoT steps that need to be verified, while multiagent debate simply copies the output of other LLMs, making it difficult to capture subtle numerical changes in long texts. A specific example is shown in Fig. 12.

This section further explores the impact of the number of agents K on the prediction performance of the TRACE framework. Fig. 13 presents the confusion matrix of the basic agents'

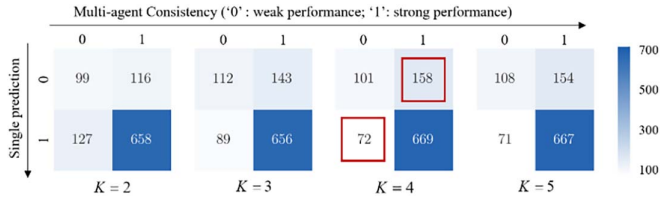


Fig. 13. Confusion matrix of MaC under different K (number of base predictors).

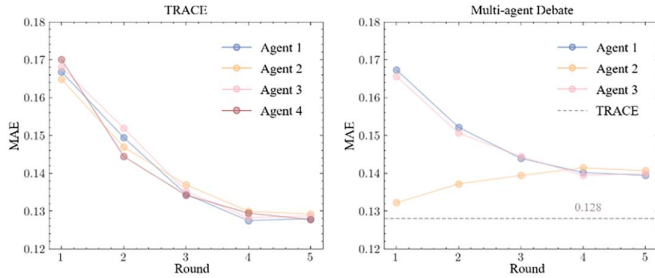


Fig. 14. Typical patterns of base predictors' MAE change with feedback round increases.

prediction performance after being processed by MaC. It can be observed that with only two agents, the number of instances where the prediction performance improves after thought refinement is 116, while the number of instances where it decreases is 127. When $K = 4$, the number of improvements increases to 158, and the number of declines decreases to 72. Based on these results, the main experiment in this article uses a base predictor count of 4.

Fig. 14 illustrates the typical performance patterns of each base predictor in TRACE and multiagent debate frameworks as the number of feedback rounds increases. It can be observed that in the TRACE framework, due to the guidance of the Judge Agent, the prediction errors of each base predictor decrease synchronously. By the fourth round of thought refinement, the prediction errors stabilize. In the multiagent debate framework, the prediction errors of each agent exhibit a typical “compromise” pattern. This may occur because even base predictors that initially provide accurate predictions begin to incorporate “inaccurate information” from other base predictors, resulting in a slight increase in prediction errors as the number of rounds increases.

4) *Potential Data Contamination Check:* To ensure the fairness of the experiment, potential data contamination check is essential. LLMs are exposed to vast amounts of data and corpora during the pretraining process. If the datasets used in this study overlaps with the pretraining data due to factors such as online dissemination, the excellent predictive performance demonstrated by TRACE could become unreliable.

We use guided instructions [36] to prompt LLMs with questions related to the three datasets used in this study, such as “What data are described in the NEST dataset?” and “What features do it have?” The ROUGE-L and BLEURT metrics are then calculated, as shown in Table VIII. ROUGE-L close to 0 indicates that the description generated by LLMs has very low

TABLE VIII
DATA CONTAMINATION CHECK

	SDWPF		Emobpy		NEST	
	ROUGE-L	BLEURT	ROUGE-L	BLEURT	ROUGE-L	BLEURT
Llama	0.1044	−0.9997	0.1298	−0.9935	0.1126	−0.9703
gpt-4o	0.1250	−0.9621	0.1304	−0.9982	0.1711	−0.8920

lexical overlap with the official dataset description. BLEURT close to -1 suggests that the LLMs' understanding of the dataset has very low semantic similarity compared to the official description. These results indicate that the dataset used in this article was unseen during the pretraining process, ensuring the authenticity and validity of this study.

V. CONCLUSION

In this study, we introduce TRACE, a novel time series forecasting framework based on LLMs CoT reasoning and MaC. TRACE is particularly well suited for DERs power forecasting scenarios that require model generalization and FSL capabilities. Future research will focus on the following areas.

- 1) This article demonstrates the potential of applying LLMs to power prediction in power systems. Future work could explore combining LLMs from different families, such as GPT, Llama, Davinci, and Deepseek, to investigate the impact of model type and complexity on prediction performance.
- 2) Future work could focus on designing more sophisticated CoT frameworks that extend beyond temporal feature consideration. Specifically, task-specific prompt engineering could be developed for diverse time-series applications, including forecasting, imputation, and classification tasks.

APPENDIX A

A specific case study of EVs charging/discharging power forecasting (Univariate time series prediction with three agents, seven CoT steps):

Prompt:

This is a time series of electric vehicles charging/discharging power time series. Consider the following CoT, predict the next six values.

Step 1: Whether the time series exhibits significant seasonality?

Step 2: Calculate the seasonality.

Step 3: Whether the time series exhibits specific user behaviors?

Step 4: Determine the user behaviors.

Step 5: Whether the time series differs in weekdays and weekends?

Step 6: Calculate the baseline of weekdays and weekends.

Step 7: Whether the time series exhibits significant trends?

From May, day 2, Saturday, 11 o'clock, to May, day 12, Tuesday, 10 o'clock. The electric vehicle charging power is 0.016, 0.016, 0.087, ..., 0.032, 0.039, 0.039.

Round 1:

Agent 1: The seasonality is 12. It seems like the EVs prefer charging between 8 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.021, 0.024, 0.024, 0.024, 0.021, 0.022.

Agent 2: The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 7 AM. The baseline for weekdays is 0.040, the baseline for weekends is 0.034. My prediction is 0.024, 0.025, 0.025, 0.025, 0.025, 0.021.

Agent 3: The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.028. My prediction is 0.021, 0.025, 0.025, 0.024, 0.024, 0.024.

Judge Agent:

Summary 1: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.032. Please check the seasonality you calculated. Would you like to refine your thought?

Summary 2: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.032. Please check the user behaviors you considered. Would you like to refine your thought?

Summary 1: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.032. Please check the baseline for weekends you calculated. Would you like to refine your thought?

Round 2:

Agent 1: Yes! After refining my thought process, I correct my answer. The seasonality is 24. It seems like the EVs prefer charging between 8 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.021, 0.024, 0.024, 0.024, 0.024, 0.021.

Agent 2: Yes! After refining my thought process, I correct my answer. The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.024, 0.025, 0.025, 0.024, 0.024, 0.024.

Agent 3: Yes! After refining my thought process, I correct my answer. The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.032. My prediction is 0.021, 0.025, 0.025, 0.024, 0.024, 0.024.

Judge Agent:

Summary 1: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.034. Please check the user behaviors you considered. Would you like to refine your thought?

Summary 2: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer

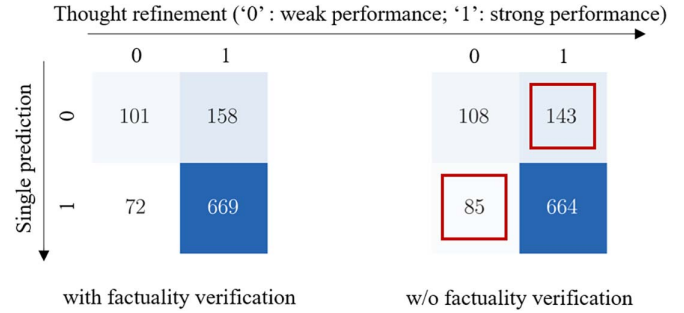


Fig. 15. Confusion matrix of thought refinement enhancement without factuality verification.

charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.034. Please check the baseline for weekends you calculated. Would you like to refine your thought?

Summary 3: According to other agents' prediction, the average seasonality is 24. Other agents agree that EVs prefer charging between 11 PM and 8 AM. The average baseline for weekdays is 0.041, the baseline for weekends is 0.034. Please check the baseline for weekends you calculated. Would you like to refine your thought?

Round 3:

Agent 1: Yes! After refining my thought process, I correct my answer. The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.021, 0.025, 0.025, 0.025, 0.024, 0.021.

Agent 2: No! I insist my answer. The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.024, 0.025, 0.025, 0.024, 0.024, 0.024.

Agent 3: No! I insist my answer. The seasonality is 24. It seems like the EVs prefer charging between 11 PM and 8 AM. The baseline for weekdays is 0.041, the baseline for weekends is 0.035. My prediction is 0.021, 0.025, 0.025, 0.024, 0.024, 0.024.

Judge Agent:

The final prediction is 0.022, 0.025, 0.025, 0.024, 0.024, 0.023.

APPENDIX B

To further validate the necessity of our Factuality Verification module (in Section III-D), we conducted an ablation study (see Table IX and Fig. 15). In this experiment, we removed the verification module entirely—each Judge Agent generated only a single summary per round without undergoing lexical overlap, semantic consistency, or factual consistency checks, and this unverified summary was directly returned to all Agents.

Across 1000 test cases, this ablated version exhibited a consistent increase in MAE per round. More critically, as revealed by the enhancement confusion matrix, the absence of verification allowed biased and erroneous summaries to propagate through the system.

TABLE IX
EACH ROUNDS MAE USING SUMMARIES WITHOUT
FACTUALITY VERIFICATION

	Round 1	Round 2	Round 3	Round 4
With factuality verification	0.167	0.149	0.136	0.129
W/o factuality verification	0.167	0.152	0.140	0.135

The ablation results clearly demonstrate that the absence of the Factuality Verification module leads to consistently elevated MAE values across each round. Analysis of the enhancement confusion matrix reveals that the unchecked propagation of biased and erroneous summaries directly contributes to this performance degradation, as evidenced by the reduced number of prediction Agents showing improved outputs after refinement. This deterioration mechanism fundamentally undermines the MaC framework's core functionality.

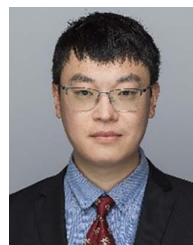
REFERENCES

- [1] A. Srivastava et al., "Distribution system behind-the-meter DERs: Estimation, uncertainty quantification, and control," *IEEE Trans. Power Syst.*, vol. 40, no. 1, pp. 1060–1077, Jan. 2025.
- [2] C. Wang, Y. Zhou, Q. Wen, and Y. Wang, "Improving load forecasting performance via sample reweighting," *IEEE Trans. Smart Grid*, vol. 14, no. 4, pp. 3317–3320, Jul. 2023.
- [3] L. Tang et al., "Privacy-preserving knowledge sharing for few-shot building energy prediction: A federated learning approach," *Appl. Energy*, vol. 337, 2023, Art. no. 120860.
- [4] OpenAI. "Introducing ChatGPT," Nov. 30, 2022. [Online]. Available: <https://openai.com/blog/chatgpt>
- [5] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco, CA, USA: Holden-Day, 1970.
- [6] R. Hyndman, A. B. Koehler, J. K. Ord, and R. D. Snyder, *Forecasting with Exponential Smoothing: The State Space Approach*, vol. 3. Springer Science & Business Media, 2008, p. 9.
- [7] L. Mochurad and A. Dereviannyi, "An ensemble approach integrating LSTM and ARIMA models for enhanced financial market predictions," *R. Soc. Open Sci.*, vol. 11, no. 9, Sep. 2024, Art. no. 240699.
- [8] L. Zhang, P. Shi, X. Lai, P. Du, F. Wen, and Y. Liu, "Air-conditioning load forecasting based on seasonal decomposition and ARIMA model," in *Proc. Int. Conf. Advance Comput. Innovative Technol. Eng. (ICACITE)*, Greater Noida, India, 2021, pp. 664–669.
- [9] K. Dab et al., "Uncertainty quantification in load forecasting for smart grids using non-parametric statistics," *IEEE Access*, vol. 12, pp. 138000–138017, 2024.
- [10] O. T. Tshipata, D. T. Kazumba, P. S. Nzakuna, V. Paciello, and A. K. Lusala, "Multi-horizon short-term electrical load forecasting: A comparative analysis of statistical models and deep neural networks," in *Proc. IEEE Int. Symp. Meas. Netw. (M&N)*, Rome, Italy, 2024, pp. 1–6.
- [11] A. Graves, "Long short-term memory," in *Supervised Sequence Labelling with Recurrent Neural Networks*, Studies in Computational Intelligence, vol. 385. Berlin, Heidelberg: Springer, 2012.
- [12] I. Sutskever, "Sequence to sequence learning with neural networks," 2014, *arXiv:1409.3215*.
- [13] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," Banff, Canada, 2013.
- [14] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, pp. 11106–11115, 2021.
- [15] H. Wu et al., "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 22419–22430.
- [16] T. Zhou et al., "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2022, pp. 27268–27286.
- [17] H. Zhao, Y. Wu, L. Ma, and S. Pan, "Spatial and temporal attention-enabled transformer network for multivariate short-term residential load forecasting," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.
- [18] Q.-T. Phan, Y.-K. Wu, and Q.-D. Phan, "Enhancing one-day-ahead probabilistic solar power forecast with a hybrid transformer-LUBE model and missing data imputation," *IEEE Trans. Ind. Appl.*, vol. 60, no. 1, pp. 1396–1408, Jan./Feb. 2024.
- [19] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [20] I. Bae, J. Lee, and H. G. Jeon, "Can language beat numerical regression? Language-based multimodal trajectory prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 753–766.
- [21] H. Xue and F. D. Salim, "PromptCast: A new prompt-based learning paradigm for time series forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6851–6864, Nov. 2024.
- [22] A. F. Ansari et al., "Chronos: Learning the language of time series," 2024, *arXiv:2403.07815*.
- [23] N. Gruver et al., "Large language models are zero-shot time series forecasters," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, p. 36.
- [24] C. Chang et al., "LLM4TS: Aligning pre-trained LLMS as data-efficient time-series forecasters," 2024, *arXiv:2308.08469*.
- [25] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [26] H. Manikandan, Y. Jiang, and J. Z. Kolter, "Language models are weak learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 50907–50931.
- [27] S. Feng et al., "Knowledge card: Filling LLMs knowledge gaps with plug-in specialized language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [28] Q. Chen et al., "Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought," in *Proc. 38th Annu. Conf. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 54872–54904.
- [29] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [30] P. Heer et al., "Comprehensive energy demand and usage data for building automation," *Sci. Data*, vol. 11, no. 1, p. 469, 2024.
- [31] J. Zhou et al., "SDWPF: A dataset for spatial dynamic wind power forecasting over a large turbine array," *Sci. Data*, vol. 11, no. 1, p. 649, 2024.
- [32] C. Gaete-Morales et al., "An open tool for creating battery-electric vehicle time series from empirical data, emobpy," *Sci. Data*, vol. 8, no. 1, p. 152, 2021.
- [33] R. Trivedi et al., "Comprehensive dataset on electrical load profiles for energy community in Ireland," *Sci. Data*, vol. 11, no. 1, p. 621, 2024.
- [34] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023.
- [35] Y. Du et al., "Improving factuality and reasoning in language models through multiagent debate," in *Proc. 41st Int. Conf. Mach. Learn. PMLR*, 2024, pp. 11733–11763.
- [36] S. Golchin and M. Surdeanu, "Time travel in LLMs: Tracing data contamination in large language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [37] P. Jeha et al., "PSA-GAN: Progressive self attention GANs for synthetic time series," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022.
- [38] X. Yuan and Y. Qiao, "Diffusion-TS: Interpretable diffusion for general time series generation," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.



Yuxuan Chen received the B.S. and M.S. degrees in 2020 and 2023, respectively, from the School of Electrical Engineering, Xi'an Jiaotong University, Xi'an, China, where he is currently working toward the Ph.D. degree, all in electrical engineering.

His research interests include time series analysis and artificial intelligence applications in power systems.



Haipeng Xie (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the School of Electrical Engineering, Xi'an Jiaotong University, Xi'an, China, in 2012 and 2019, respectively, all in electrical engineering.

Currently, he is an Associate Professor with the School of Electrical Engineering, Xi'an Jiaotong University. His research interests include resilience of energy systems, quantum informatics in power systems, and artificial intelligence applications in power systems.